Reporte

Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Implementación del núcleo TechStore
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Millett, S., & Tune, N. (2015). *Patterns, principles, and practices of domain-driven design*. Wrox.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley Professional.

Equipo 6

Jesús Eduardo Moreno Osoria	**2136063**
Osvaldo Salvador Cárdenas Mejorado	**2141553**

Objetivo:

El estudiante aplicará técnicas formales de levantamiento y documentación de requerimientos de negocio, construyendo el BRD, User Stories y modelos BPMN de un caso real asignado.

Reflexión sobre los Errores Encontrados

TechStore – Actividad Práctica 4

Durante el desarrollo encontramos varios problemas; aquí documentamos los tres que más nos enseñaron. Para cada uno indicamos qué pasó, cómo nos dimos cuenta, cuál era la causa, cómo lo solucionamos, cómo comprobamos la solución y qué aprendimos.

Error 1. El estado del pedido se podía cambiar desde fuera

Aspecto	Descripción
Qué pasó	En la primera versión, Order era una clase de datos con atributos públicos, como en el ejemplo de la guía. Cualquier parte del programa podía escribir order.status = "SHIPPED" o hacer order.items.append(...).
Cómo lo detectamos	Al comparar el código de ejemplo de la guía con la regla de las 20 piezas: con una clase de datos abierta, order.items.append(OrderItem("SKU003", 50, ...)) agregaría 50 piezas sin pasar por add_item(), que es justo donde se revisa el límite.
Causa	Las reglas estaban en los métodos, pero los datos seguían abiertos: había una «puerta trasera» que evitaba los métodos.
Solución	status, items y events se guardan en atributos privados y se exponen como propiedades de solo lectura; items se devuelve como tupla, que no se puede modificar.
Cómo lo comprobamos	test_status_no_se_puede_asignar_desde_fuera y test_items_no_se_pueden_modificar_desde_fuera intentan esas asignaciones y verifican que fallen.
Aprendizaje	El Aggregate Root debe ser la única puerta para modificar sus datos; si no, las reglas se pueden saltar.

Error 2. Los mensajes mostraban códigos internos

Aspecto	Descripción
Qué pasó	Los mensajes de error decían, por ejemplo, «No se puede confirmar un pedido en estado CONFIRMED». Además, si en Swagger se escribía «dos» en lugar de un número, FastAPI respondía en inglés: «Input should be a valid integer…».
Cómo lo detectamos	Por la observación que hizo el profesor en las revisiones: el cliente no tiene por qué saber qué significa DRAFT o CONFIRMED. Después revisamos todos los mensajes que puede devolver la API.

Aspecto	Descripción
Causa	Usábamos el mismo texto para el programador y para el cliente: el código interno del estado aparecía dentro del mensaje, y los errores de formato los generaba FastAPI con su texto predeterminado.
Solución	Cada estado tiene una etiqueta en español («Pendiente de confirmar», «Enviado»…); los mensajes se redactan según la situación («…porque ya fue enviado»); un manejador traduce los errores de formato; la respuesta separa error (código para programas) de mensaje (texto para la persona).
Cómo lo comprobamos	test_mensajes_de_error_sin_codigos_internos provoca todos los tipos de error y falla si algún mensaje contiene DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED o RETURNED.
Aprendizaje	El lenguaje interno del sistema y el lenguaje del cliente son distintos; ambos pueden convivir sin mezclarse si se separan desde el diseño.

Error 3. El total de un pedido vacío aparecía como 0

Aspecto	Descripción
Qué pasó	Al ejecutar python cargar_csv.py, todos los pedidos mostraban su total con dos decimales (por ejemplo 2100.00), excepto O007, el pedido vacío, que mostraba solo 0.
Cómo lo detectamos	Revisando la salida de la carga contra la tabla que habíamos calculado a mano.
Causa	Money.zero() creaba el importe con Decimal("0"). Un Decimal conserva la cantidad de decimales con la que se crea, así que el cero no tenía centavos.
Solución	Crear el cero como Decimal("0.00").
Cómo lo comprobamos	Volvimos a ejecutar la carga: O007 muestra 0.00 y la API devuelve "total": "0.00".
Aprendizaje	Con dinero importan el valor y la precisión; por eso los importes se crean desde texto (Decimal("600.00")) y se redondean a centavos.

Lo que aprendimos como equipo

Aprendimos a escribir primero la regla y su prueba, y después el código; y que la evidencia de calidad no es que el programa arranque, sino que las reglas se cumplan y que las pruebas fallen cuando alguien las rompe. Para comprobarlo introdujimos a propósito seis errores en el código, y las pruebas los detectaron todos.